

```
In [ ]: EDA_Assignment_Ankit Senger
```

```
In [74]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [75]: df = pd.read_csv(r"D:\Data Science\Assignment\EDA Assignment\eda_assignment.csv")
```

```
In [76]: df.head()
```

```
Out[76]:
```

	Jersey No	Player	Matches	Inns	Not Out	Runs	Highest Score	Avg	Balls faced	Strike rate	100	50	4s	6s
0	1	KL Rahul	14	14	2	670	132*	55.83	518	129.34	1	5	58	23
1	2	Shikhar Dhawan	17	17	3	618	106*	44.14	427	144.73	2	4	67	12
2	3	David Warner	16	16	2	548	85*	39.14	407	134.64	0	4	52	14
3	4	Shreyas Iyer	17	17	2	519	88*	34.60	421	123.27	0	3	40	16
4	5	Ishan Kishan	14	13	4	516	99	57.33	354	145.76	0	4	36	30

```
In [77]: df.tail()
```

```
Out[77]:
```

	Jersey No	Player	Matches	Inns	Not Out	Runs	Highest Score	Avg	Balls faced	Strike rate	100	50	4s	6s
128	129	Khaleel Ahmed	7	1	0	0	0*	0.0	2	0.0	0	0	0	0
129	130	Arshdeep Singh	8	1	0	0	0*	0.0	3	0.0	0	0	0	0
130	131	Daniel Sams	3	1	0	0	0*	0.0	2	0.0	0	0	0	0
131	132	Shreevats Goswami	2	2	0	0	0*	0.0	4	0.0	0	0	0	0
132	133	Trent Boult	15	1	0	0	0*	0.0	1	0.0	0	0	0	0

In []: Q1. What **is** the maximum number of matches played by an individual player **in** a season?
Print the player name along **with** the number of matched played.

```
In [100... max_matches = df['Matches'].max()
print(max_matches)
```

17

```
In [101... max_matches = df[df['Matches'] == max_matches][['Player', 'Matches']]
print(max_matches)
```

	Player	Matches
1	Shikhar Dhawan	17
3	Shreyas Iyer	17
19	Marcus Stoinis	17
67	Kagiso Rabada	17

In []: Q2. Top 2 players **with** maximum Average who have scored atleast 2 half centuries ?

```
In [102... half_century = df[df["50"] >= 2]
top_2avg_players = half_century.nlargest(2, "Avg")[["Player", "Avg"]]
print(top_2avg_players)
```

	Player	Avg
36	Wriddhiman Saha	71.33
4	Ishan Kishan	57.33

In []: Q3. Create 2 new columns based on Player name.
First column will have first name **and** second column will have last name.
Eg: **for** the player Shikhar Dhawan, Shikhar will be the first name **and** Dhawan will be the last name.

```
In [103... df[["First Name", "Last Name"]] = df["Player"].str.split(n=1, expand=True)
print(df[["Player", "First Name", "Last Name"]])
```

	Player	First Name	Last Name
0	KL Rahul	KL	Rahul
1	Shikhar Dhawan	Shikhar	Dhawan
2	David Warner	David	Warner
3	Shreyas Iyer	Shreyas	Iyer
4	Ishan Kishan	Ishan	Kishan
..	...	...	...
128	Khaleel Ahmed	Khaleel	Ahmed
129	Arshdeep Singh	Arshdeep	Singh
130	Daniel Sams	Daniel	Sams
131	Shreevats Goswami	Shreevats	Goswami
132	Trent Boult	Trent	Boult

[133 rows x 3 columns]

```
In [ ]: Q4. Create a new column (Cleaned_Highest_score) based on Highest score variable.
        Remove the Asterik(*) mark and convert the data type into INT.
```

```
In [104... df["Clean_Highest_score"] = df["Highest Score"].str.replace("*", '').astype(int)
print(df[["Player", "Highest Score", "Clean_Highest_score"]])
```

	Player	Highest Score	Cleaned_Highest_score
0	KL Rahul	132*	132
1	Shikhar Dhawan	106*	106
2	David Warner	85*	85
3	Shreyas Iyer	88*	88
4	Ishan Kishan	99	99
..	...	...	...
128	Khaleel Ahmed	0*	0
129	Arshdeep Singh	0*	0
130	Daniel Sams	0*	0
131	Shreevats Goswami	0*	0
132	Trent Boult	0*	0

[133 rows x 3 columns]

```
In [ ]: Q5. Print the total number of centuries scored in the entire season.
```

```
In [105... total_centuries = df["100"].sum()
print(total_centuries)
```

5

```
In [ ]: Q6. Print all the player names whose strike rate is less than the average strike rate of all players in entire season.  
        Print the player name, his strike rate and average strike rate.
```

```
In [106... average_strike_rate = df["Strike rate"].mean()  
players_name = df[df["Strike rate"] < average_strike_rate][["Player", "Strike rate"]]  
players_name["Average Strike Rate"] = average_strike_rate  
print(average_strike_rate)  
print(players_name)
```

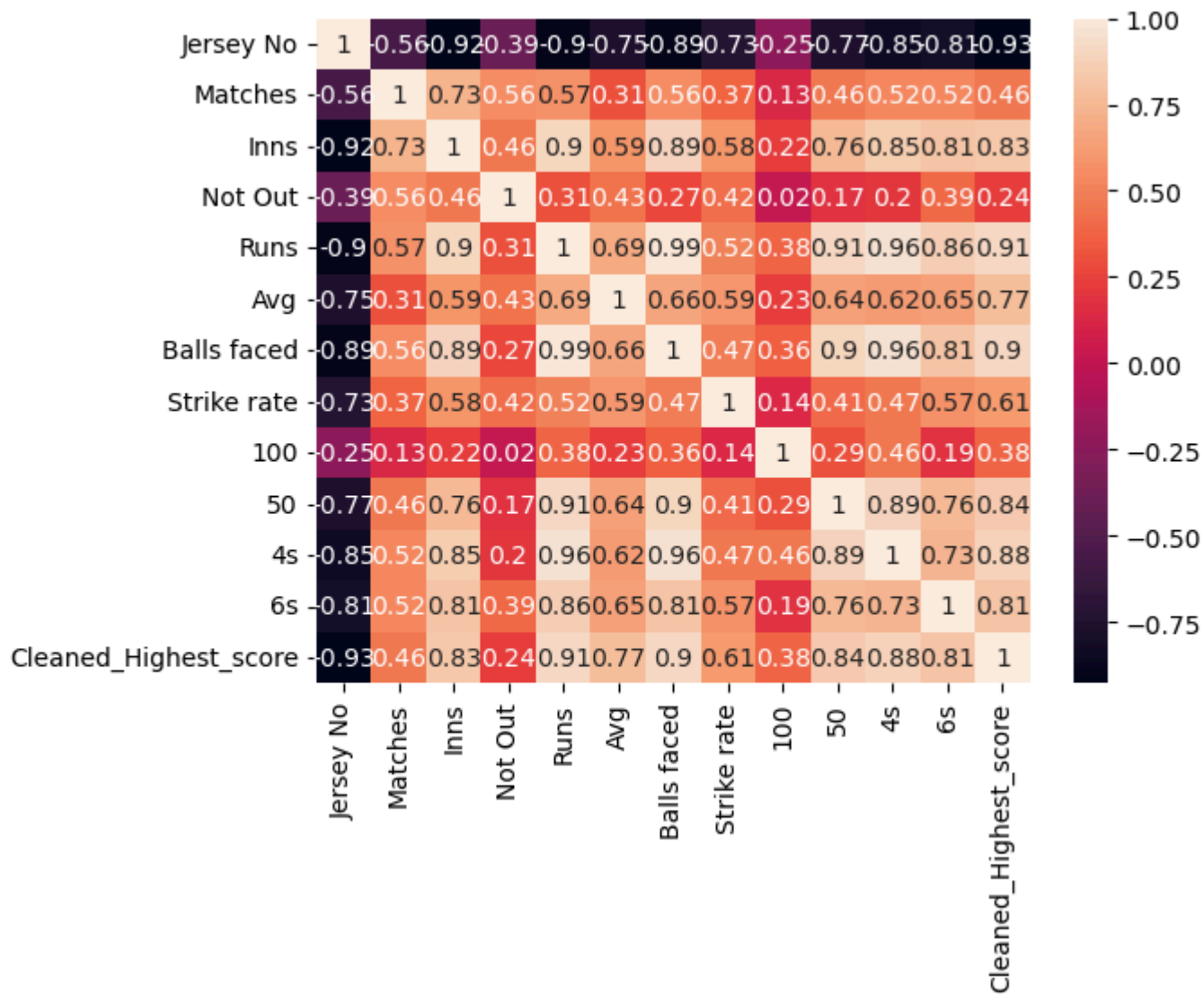
107.36473684210526

	Player	Strike rate	Average Strike Rate
51	Ajinkya Rahane	105.60	107.364737
55	Glenn Maxwell	101.88	107.364737
58	Vijay Shankar	101.04	107.364737
61	Josh Philippe	101.29	107.364737
62	Gurkeerat Singh	88.75	107.364737
65	Kedar Jadhav	93.93	107.364737
70	Yashasvi Jaiswal	90.90	107.364737
71	Shreyas Gopal	94.87	107.364737
77	Murali Vijay	74.41	107.364737
79	Chris Jordan	93.54	107.364737
80	Navdeep Saini	100.00	107.364737
82	Kamlesh Nagarkoti	70.96	107.364737
84	Harshal Patel	87.50	107.364737
85	Jimmy Neesham	105.55	107.364737
86	Tom Banton	90.00	107.364737
89	Prabhsimran Singh	100.00	107.364737
92	Kuldeep Yadav	61.90	107.364737
94	Moeen Ali	75.00	107.364737
95	Sandeep Sharma	80.00	107.364737
96	Shardul Thakur	57.14	107.364737
98	Rinku Singh	100.00	107.364737
99	Shivam Mavi	71.42	107.364737
100	Varun Chakaravarthy	66.66	107.364737
101	Jaydev Unadkat	69.23	107.364737
102	Ankit Rajpoot	90.00	107.364737
104	Shahbaz Nadeem	87.50	107.364737
105	Pravin Dubey	53.84	107.364737
107	Deepak Chahar	58.33	107.364737
108	Ravi Bishnoi	58.33	107.364737
109	Andrew Tye	100.00	107.364737
111	Kartik Tyagi	66.66	107.364737
112	Murugan Ashwin	100.00	107.364737
114	T Natarajan	60.00	107.364737
115	Prasidh Krishna	50.00	107.364737
116	Rahul Chahar	50.00	107.364737
117	Mohammad Shami	66.66	107.364737
118	Nikhil Naik	33.33	107.364737
119	Mujeeb Ur Rahman	33.33	107.364737
120	Dale Steyn	50.00	107.364737

121	Varun Aaron	10.00	107.364737
122	Shahbaz Ahmed	100.00	107.364737
123	Yuzvendra Chahal	33.33	107.364737
124	Mitchell Marsh	0.00	107.364737
125	Umesh Yadav	0.00	107.364737
126	Bhuvneshwar Kumar	0.00	107.364737
127	Sheldon Cottrell	0.00	107.364737
128	Khaleel Ahmed	0.00	107.364737
129	Arshdeep Singh	0.00	107.364737
130	Daniel Sams	0.00	107.364737
131	Shreevats Goswami	0.00	107.364737
132	Trent Boult	0.00	107.364737

```
In [ ]: Q7. Please check the correlation between the features and create a heat map.
```

```
In [88]: sns.heatmap(df.corr(numeric_only=True), annot=True)  
plt.show()
```



In []: Q8. Check the list of players who has an average greater than 50 as well strike rate above 120. Print player name, average and strike rate.

```
In [107... players = df[(df["Avg"] > 50) & (df["Strike rate"] > 120)][["Player", "Avg", "Strike rate"]]
print(players)
```

	Player	Avg	Strike rate
0	KL Rahul	55.83	129.34
4	Ishan Kishan	57.33	145.76
31	Kieron Pollard	53.60	191.42
36	Wriddhiman Saha	71.33	139.86
37	Ruturaj Gaikwad	51.00	120.71
57	Deepak Hooda	101.00	142.25
60	Tom Curran	83.00	133.87

In []: Q9. Please check the list of players who has an average greater than 40 and balls faced above 100.
Print player name, average and balls faced.

```
In [108... players_40 = df[(df["Avg"] > 40) & (df["Balls faced"] > 100)][["Player", "Avg", "Balls faced"]]
print(players_40)
```

	Player	Avg	Balls faced
0	KL Rahul	55.83	518
1	Shikhar Dhawan	44.14	427
4	Ishan Kishan	57.33	354
8	Virat Kohli	42.36	384
9	ABD Villiers	45.40	286
10	Faf Duplessis	40.81	319
14	Eoin Morgan	41.80	302
24	Kane Williamson	45.28	237
27	Chris Gayle	41.14	210
28	Ben Stokes	40.71	200
31	Kieron Pollard	53.60	140
32	Rahul Tewatia	42.50	183
33	Ravindra Jadeja	46.40	135
36	Wriddhiman Saha	71.33	153
37	Ruturaj Gaikwad	51.00	169

In []: Q10. Players who scored atleast one century in this season. Create visualization.

```
In [109... players_centuries = df[df["100"] > 0][["Player", "100"]]
print(players_centuries)
```

	Player	100
0	KL Rahul	1
1	Shikhar Dhawan	2
13	Mayank Agarwal	1
28	Ben Stokes	1

In []: Q11. Players who scored atleast 4 half centuries in this season.

```
In [91]: players_50 = df[df["50"] >= 4][["Player", "50"]]
print(players_50)
```

	Player	50
0	KL Rahul	5
1	Shikhar Dhawan	4
2	David Warner	4
4	Ishan Kishan	4
5	Quinton Kock	4
6	Suryakumar Yadav	4
7	Devdutt Padikkal	5
9	ABD Villiers	5
10	Faf Duplessis	4

In []: Q12. Check the list of players who hit more than 45 boundaries and more than 10 sixes in this season.

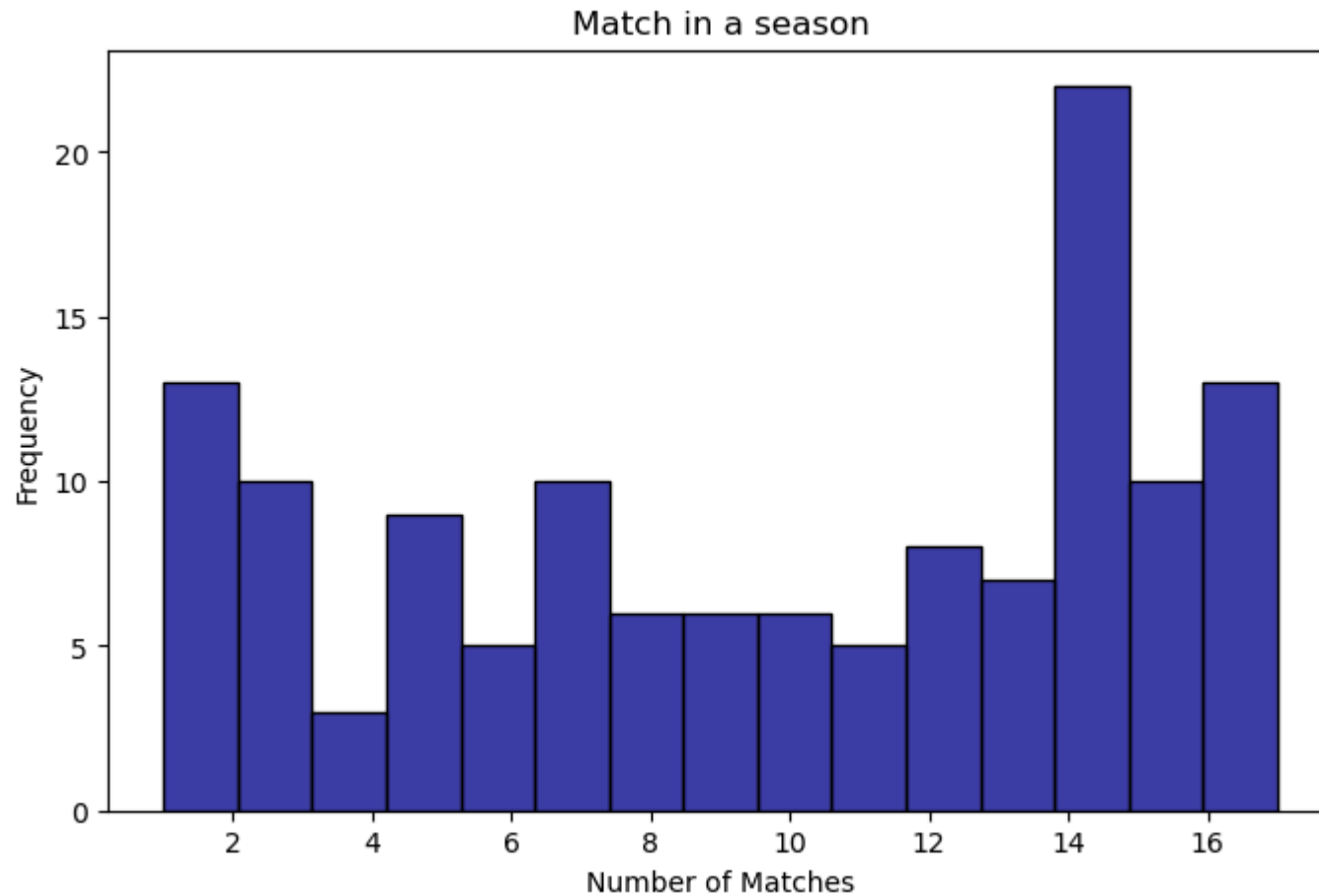
```
In [110]: players_4and6 = df[(df["4s"] > 45) & (df["6s"] > 10)][["Player", "4s", "6s"]]
print(players_4and6 )
```

	Player	4s	6s
0	KL Rahul	58	23
1	Shikhar Dhawan	67	12
2	David Warner	52	14
5	Quinton Kock	46	22
6	Suryakumar Yadav	61	11

In []: Q13. Plot a histogram of number of matches played in a season by players.

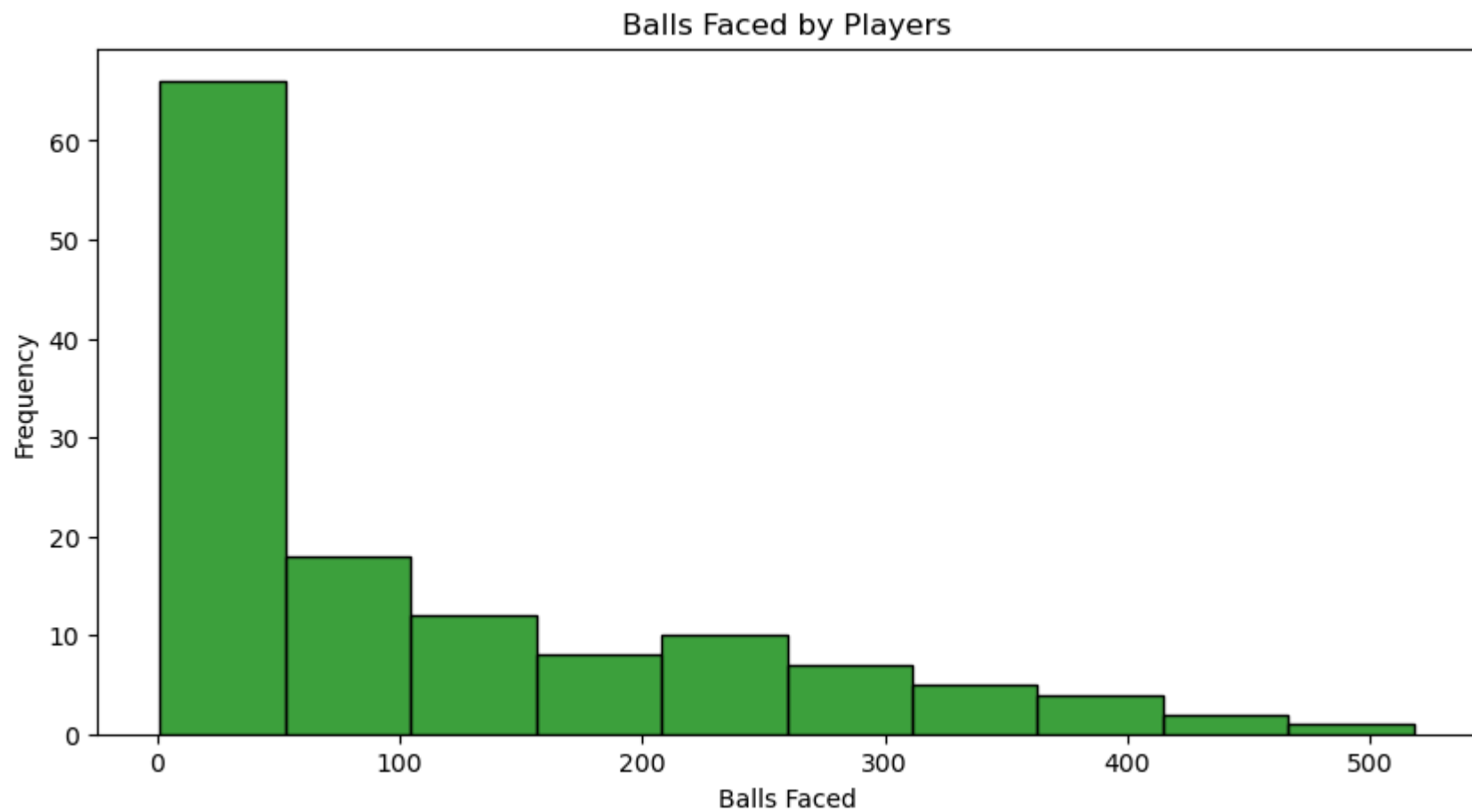
```
In [111]: plt.figure(figsize=(8, 5))
sns.histplot(df["Matches"], bins=15, color="darkblue")
plt.title("Match in a season")
plt.xlabel("Number of Matches")
```

```
plt.ylabel("Frequency")
plt.show()
```



In []: Q14. Plot the histogram of balls faced by players.

```
In [48]: plt.figure(figsize=(10, 5))
sns.histplot(df["Balls faced"], bins=10, color="green")
plt.title("Balls Faced by Players")
plt.xlabel("Balls Faced")
plt.ylabel("Frequency")
plt.show()
```



In []: Q15. Top 10 players with most runs in a season.

```
In [112... players_runs = df.nlargest(10, "Runs")[["Player", "Runs"]]  
print(players_runs)
```

	Player	Runs
0	KL Rahul	670
1	Shikhar Dhawan	618
2	David Warner	548
3	Shreyas Iyer	519
4	Ishan Kishan	516
5	Quinton Kock	503
6	Suryakumar Yadav	480
7	Devdutt Padikkal	473
8	Virat Kohli	466
9	ABD Villiers	454

In []: Q16. Print the players who played the match but didn't get the batting.

```
In [113... no_batting = df[df["Runs"] == 0][["Player", "Matches"]]
print(no_batting)
```

	Player	Matches
124	Mitchell Marsh	1
125	Umesh Yadav	2
126	Bhuvneshwar Kumar	4
127	Sheldon Cottrell	6
128	Khaleel Ahmed	7
129	Arshdeep Singh	8
130	Daniel Sams	3
131	Shreevats Goswami	2
132	Trent Boult	15

In []: Q17. Create a new column to show the percentage of total runs scored in 4s and 6s.
Then print the top 5 players with maximum percentage.

```
In [56]: df["% Runs in 4s"] = ((df["4s"]*4) / df["Runs"])* 100
df["% Runs in 6s"] = ((df["6s"]*4) / df["Runs"])* 100
print(df["% Runs in 4s"])
print(df["% Runs in 6s"])
```

```

0      34.626866
1      43.365696
2      37.956204
3      30.828516
4      27.906977
...
128      NaN
129      NaN
130      NaN
131      NaN
132      NaN
Name: % Runs in 4s, Length: 133, dtype: float64
0      13.731343
1       7.766990
2     10.218978
3     12.331407
4     23.255814
...
128      NaN
129      NaN
130      NaN
131      NaN
132      NaN
Name: % Runs in 6s, Length: 133, dtype: float64

```

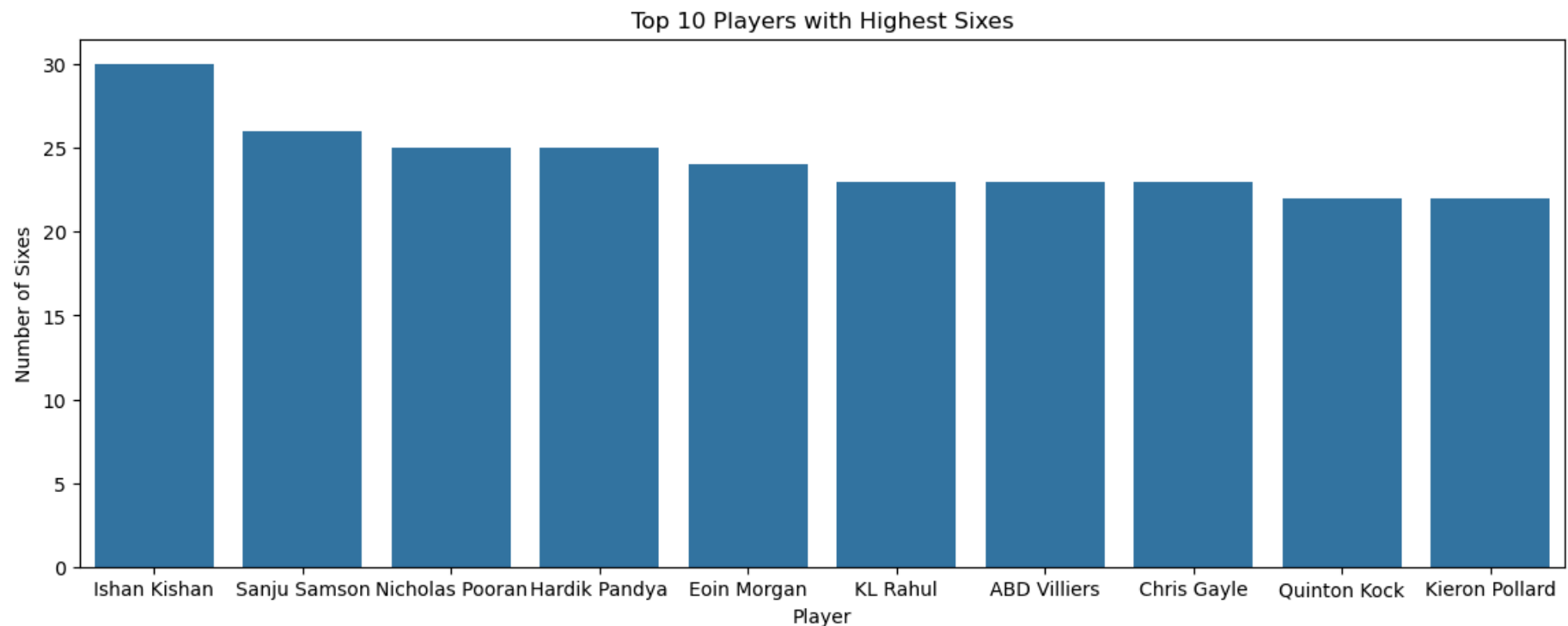
```
In [ ]: Q18. Print the players with top 5 Not out percentages
        (Not Out percentage can be calculated as number of Not outs divided by Innings).
```

```
In [61]: df["NotOut%"] = (df["Not Out"] / df["Matches"]) * 100
        top_5 = df.nlargest(5, "NotOut%")["Player", "NotOut%"]
        print(top_5)
```

	Player	NotOut%
97	Mohammad Nabi	100.000000
113	Dhawal Kulkarni	100.000000
62	Gurkeerat Singh	62.500000
60	Tom Curran	60.000000
57	Deepak Hooda	57.142857

```
In [ ]: Q19. Create visualization of top 10 players with highest number of sixes.
```

```
In [69]: top_10_sixes = df.nlargest(10, "6s")[["Player", "6s"]]
plt.figure(figsize=(14, 5))
sns.barplot(x="Player", y="6s", data=top_10_sixes)
plt.title("Top 10 Players with Highest Sixes")
plt.xlabel("Player")
plt.ylabel("Number of Sixes")
plt.show()
```



In []: Q20. Scatter plot of runs scored by a player v/s balls faced in a season.
Then find the relationship between these 2 variables.

```
In [73]: plt.figure(figsize=(10, 8))
sns.scatterplot(x="Balls Faced", y="Runs", data=df, color="darkred")
plt.title("Runs Scored vs Balls Faced")
plt.xlabel("Balls Faced")
plt.ylabel("Runs Scored")
plt.show()
```

